Final Project Report for QuickTask

1. Description

QuickTask is a simplistic, yet full-stack web application designed to solve the problem of exited from the to dos. It minimizes anxiety and feeling of being overwhelmed by long master list, yet boosts productivity and progress by measuring and analyzing energy level and available time .

How This project is different ?

Standard productivity apps usually overwhelm users with giant, stressful lists of tasks. QuickTask fixes this by only showing a maximum of three actionable tasks at a time based on a user's current energy level (High or Low). To meet the provided course rubric criteria, the app automatically collects data, analyzes it and fetches random activity ideas from an external REST API, processes them on the backend, and saves them directly into a persistent database for safe storage.

2. System diagram:

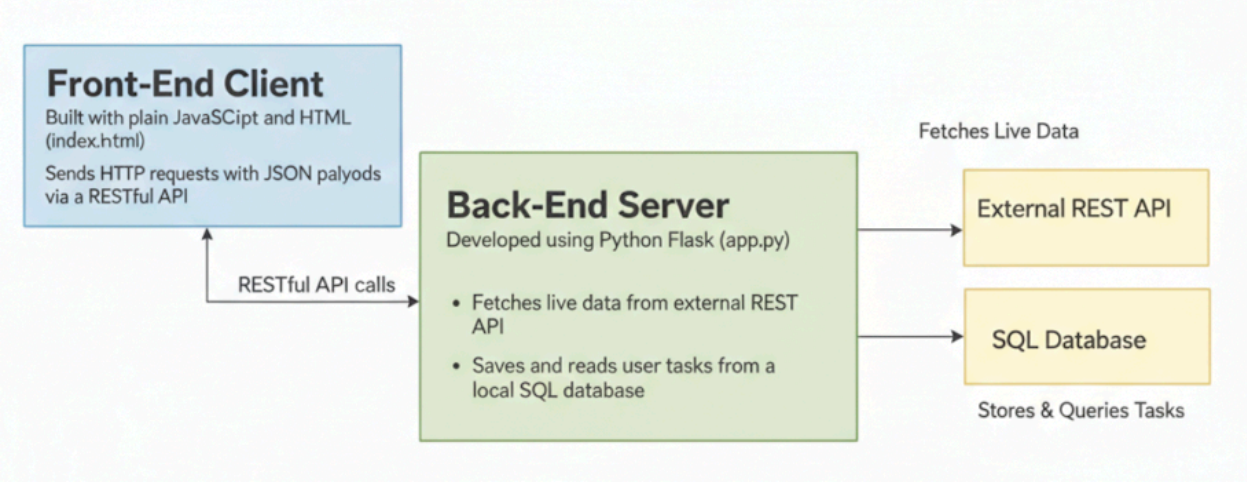
Front-End Client
- Built with plain JavaScript and HTML (index.html)
- Sends HTTP requests with JSON payloads via a RESTful API

↓ (RESTful API calls)

Back-End Server
- Developed using Python Flask (app.py)
- Handles two main functions:
  - Fetches live data from an external REST API
  - Saves and reads user tasks by querying a local SQL database

→ Fetches Live Data →
External REST API

→ Stores & Queries Tasks →
SQL Database



3. Overview of Processes and Services

a) Front-End: A clean user interface built with plain HTML and JavaScript. It runs entirely in the user's web browser and uses asynchronous fetch()requests to talk to the backend without forcing the webpage to reload.
b) Back-End: A lightweight Python server built with Flask. It acts as the brain of the project. It handles incoming browser requests, routes data, connects to the external API, and manages database operations.
c) External Source: A public, third-party REST API that provides random productivity ideas in JSON format whenever our backend asks for one.
d) Data Storage: A local database instance that acts as our permanent storage, making sure our tasks stay saved even if the server restarts.

In other words, the front-end client runs directly in the user's web browser, handling the buttons and the overall screen layout. When a user clicks a button, the front-end talks to the back-end asynchronously using a RESTful API, meaning it sends and receives task data in the background without forcing the entire webpage to reload. The back-end Flask server acts as the central brain of the app, running the main logic to process data, fetching new tasks from the external public API over the internet, and saving everything to the database. Finally, the data tier uses a structured SQL database to act as our permanent storage, ensuring that user tasks are safely saved and never lost even if the web server restarts.

4. Design Decisions

I chose a relational SQL database over a NoSQL document database because our task data follows a strict, simple format with specific columns for the task ID, title, and energy level. SQL databases are excellent for this because they use structured tables and guarantee strict transactional safety, which keeps our data clean and prevents things from breaking during write cycles. By using an SQL database, the app can run quick and easy filtering queries to match a specific energy level and limit the view to 3 items, which is much faster than scanning through unstructured NoSQL JSON files. Additionally, I preferred using plain JavaScript and Python Flask instead of heavy frameworks like React and Node.js to avoid dealing with massive, bloated dependency folders and complicated build setups, allowing the app to run fast and deploy live on the cloud in under a minute.

5. System Requirements and Testability

The project meets three main requirements that are very easy to check and test using simple validation scripts. The first requirement is that the database tables must set themselves up automatically on startup, which I can test by launching the app and verifying that the database file is instantly created with the correct table layout. The second requirement is that the app must successfully fetch data from an external REST API and save it, which is testable by triggering the fetch action and making sure the total row count in our database table increases by exactly one. Lastly, the user requirement states that the app must limit the visible dashboard list to a maximum of three items, which is easily tested by loading five tasks into the database, calling the retrieval endpoint, and confirming that the returned data array length is exactly three.